



UV-Atlas Latent Dynamics Towards Sampling-Agnostic Cloth Simulation

Final Report

June 4, 2026

Student: **Antonin GRANADOS**
antonin.granados@telecom-paris.fr

Supervisor: **Kiwon UM**
kiwon.um@telecom-paris.fr

School: **Télécom Paris**
Course: **MDC-4EL02-TP (RECH-202)**

Abstract

We present a UV-atlas latent simulator for cloth dynamics. Instead of evolving cloth motion directly on mesh vertices, the method rasterises the cloth state into a fixed-resolution UV-atlas representation, encodes it into a latent atlas map, performs autoregressive dynamics in latent space, and decodes mesh positions through UV-coordinate queries. This separates the recurrent computation from the number of output vertices and reduces the dependence of the recurrent dynamics on the mesh sampling used at inference time. The architecture combines cross-atlas convolutions for local material-space processing with physics-attention blocks for longer-range interactions, and is trained with supervised rollout losses and physics-based regularisation.

1. Introduction

The simulation of deformable objects, and cloth in particular, is an important problem in computer graphics. Realistic cloth motion is used in film animation, video games, virtual reality, virtual try-on, and more generally in any application involving animated garments or flexible surfaces. Classical physics-based simulators can produce highly realistic results by solving elastic, bending, inertial, and collision constraints [1, 2, 3, 4]. However, these methods are often computationally expensive, especially for high-resolution meshes or long sequences. This makes them difficult to use in real-time or interactive settings.

Recent works have explored deep learning as an alternative to classical simulation. A common approach is to learn the dynamics of the cloth directly from simulated data. Among these methods, graph-based neural simulators have become one of the most popular families. MeshGraphNets represent the simulation state as a graph, where vertices are nodes and mesh edges encode local physical interactions [5]. This formulation is natural for mesh-based objects and has been extended to cloth simulation through hierarchical graphs, collision-aware models, and improved generalisation strategies [6, 7, 8]. These models can generate plausible cloth motion, but they still process the cloth directly on the mesh.

This creates an important limitation. The computational cost of a graph-based simulator depends on the number of vertices and edges. When the mesh resolution increases, the recurrent computation becomes more expensive. Moreover, local message passing only propagates information through a limited graph neighbourhood at each layer. Long-range interactions therefore require many message-passing steps or a hierarchical representation. This issue has also been discussed in recent mesh-based learned simulators, where transformers or physics-attention mechanisms are introduced to reduce the under-reaching problem of local message passing [9, 10].

Another limitation is the dependence on a specific mesh discretisation. In computer graphics, the same cloth surface may exist at different resolutions depending on the application, the rendering quality, or the asset pipeline. Ideally, a learned simulator should not be strongly tied to one particular vertex sampling. Recent work such as ClothTransformer addresses this problem by compressing arbitrary-resolution meshes into a fixed-size set of latent tokens, so that the temporal dynamics are computed in latent space rather than directly on the full mesh [11]. This motivates a more general idea: the recurrent dynamics of a learned cloth simulator should be separated from the number of vertices in the output mesh.

In this work, we propose a UV-atlas latent simulator for cloth dynamics. Instead of applying the recurrent model directly on mesh vertices, we rely on the mesh’s UV mapping, a standard representation in computer graphics used to attach textures and material properties to a surface. The resulting atlas is used to represent surface quantities such as positions, velocities, rest configuration, node types, and strain. The atlas observation is encoded into a latent atlas map, evolved autoregressively in latent space, and decoded back to mesh positions by querying the latent map at the UV coordinates of the output vertices.

This design is inspired by three main ideas. First, like graph-based learned simulators, we keep local physical information such as strain and use physics-based losses to improve plausibility [5, 12]. Second, like recent latent-space simulators, we aim to reduce the dependence of the recurrent dynamics on the mesh resolution [11]. Third, we use atlas convolutions and physics-attention blocks to combine local material-space processing with longer-range latent interactions [13, 9]. The recurrent transition is no longer applied on the output mesh vertices, which is the main step toward resolution-agnostic cloth simulation.

In summary, the contributions of this work are:

- We introduce a UV-atlas latent representation for learned cloth simulation, where the recurrent

state is a fixed-resolution latent map instead of a mesh graph, making the recurrent transition independent of vertex sampling.

- We propose a latent dynamics model combining local atlas convolutions with physics-attention blocks, allowing the model to process local material-space structure while propagating information over longer ranges.
- We decode mesh positions through UV queries, so the same latent atlas state can be evaluated on different mesh samplings when compatible UV coordinates are available.

2. Related Work

Physics-Based and Learned Cloth Simulation. Cloth simulation has been studied for several decades in computer graphics. Classical methods model cloth as a thin deformable surface and solve its motion using elastic energies, bending models, collision constraints, and numerical integration [1, 2]. More advanced systems such as ARCSim introduced adaptive remeshing and realistic folding and crumpling behaviour, and are still commonly used to generate synthetic cloth datasets [3, 4]. These simulators can produce high-quality results, but remain expensive for high-resolution meshes and long sequences. Learning-based methods aim to replace part of this cost with neural predictors. Early garment-specific approaches focus mostly on virtual try-on or pose-conditioned animation [14, 15, 16], while more general learned simulators model the dynamics directly from mesh states [5, 6, 7, 8, 11, 17, 18, 19].

Graph-Based Neural Simulators. Graph Neural Networks are a natural choice for mesh-based simulation because vertices can be treated as nodes and mesh edges as local physical interactions. This idea was first explored for learned physical simulation on graphs [19], and then extended to mesh-based dynamics by MeshGraphNets [5]. Several cloth simulators build on this formulation. HOOD introduces hierarchical graphs for garment dynamics [6], SENC improves self-collision handling [7], and MeshGraphNetRP studies generalisation and stability for GNN-based cloth simulation [8]. Other mesh-based cloth models such as N-Cloth and FastClothGNN also rely on graph or mesh connectivity to predict deformation efficiently [17, 18]. However, the recurrent computation of these methods remains tied to the mesh resolution: increasing the number of vertices also increases the number of nodes, edges, and message-passing operations.

Scalability, Hierarchy, and Latent Dynamics. Several works address the cost of message passing by introducing multi-scale or latent representations. MultiScale MeshGraphNets and BSMS-GNN use hierarchical graph structures to improve long-range propagation [20, 21]. These methods reduce the locality limitation of standard message passing, but require additional hierarchy or coarsening design. More recent works instead move the temporal dynamics to a compact latent space. ClothTransformer compresses arbitrary-resolution cloth meshes into a fixed-size set of latent tokens and evolves them autoregressively with a Transformer [11]. This makes the temporal computation independent of the number of mesh vertices, although encoding and decoding still depend on the mesh size. Our method follows the same motivation, but uses a fixed-resolution UV-atlas latent map.

Physics Losses. Learned simulators trained purely with data-driven supervision often suffer from drift during long rollouts. Physics-based losses reduce this problem by penalising unrealistic deformations. SNUG trains dynamic garments using self-supervised losses based on inertia, stretching, bending, gravity, and collision terms [12], while HOOD also uses physics-inspired objectives in a hierarchical graph setting [6].

Transformers, Physics-Attention, and Atlas Representations. Transformers have been introduced in learned simulation to improve long-range interaction modelling beyond local message passing. Transolver proposes physics-attention, where states are grouped into learned physical tokens before applying attention [9]; Transolver++ improves this idea with a sharper slicing mechanism [22]. MeshGraphNet-Transformer combines local MeshGraphNet-style processing with physics-attention blocks to reduce the under-reaching problem on large meshes [10]. Our processor is inspired by this combination of local and global updates. However, instead of operating directly on mesh nodes, we apply the dynamics in a UV-atlas latent space. Atlas-based representations are common in graphics, and cross-atlas convolution allows convolutional operations to respect surface neighbourhoods across UV seams [13].

3. Learning Cloth Dynamics

We consider the problem of learning the dynamics of a deformable cloth surface represented by a triangular mesh $M = (V, F)$, where V is the set of vertices and F is the set of triangular faces. At each time step t , each vertex $i \in V$ has a 3D position $x_i^{(t)} \in \mathbb{R}^3$, a rest position $\bar{x}_i \in \mathbb{R}^3$, and a type label $c_i \in \{\text{free}, \text{pinned}\}$. In our formulation, the velocity $v_i^{(t)} \in \mathbb{R}^3$ is implicitly available from the difference between positions at times t and $t - 1$.

Each vertex is also associated with a UV coordinate $u_i \in [0, 1]^2$, which maps the vertex to a point in the 2D material domain of the cloth. In practice, a vertex may have several UV coordinates when it lies on the boundary between two atlas charts. We handle this case through the atlas construction described in Section 4.1.

We use the Saint Venant-Kirchhoff material model to describe the elastic behaviour of the cloth, following common physics-based and self-supervised cloth formulations [12]. In addition to the per-vertex quantities defined above, we include the per-face Green strain tensor components E_{xx} , E_{yy} and E_{xy} in the state. These quantities are described in more detail in Section 4.1.

We denote the full geometric state at time t by

$$X^{(t)} = \{x_i^{(t)}\}_{i \in V}.$$

We denote the Green strain components of face f at time t by

$$E_f^{(t)} = (E_{xx,f}^{(t)}, E_{yy,f}^{(t)}, E_{xy,f}^{(t)}) \in \mathbb{R}^3.$$

The full cloth state is then written as

$$S^{(t)} = (\{x_i^{(t)}, v_i^{(t)}, \bar{x}_i, c_i\}_{i \in V}, \{E_f^{(t)}\}_{f \in F}).$$

The goal is to learn a simulator that predicts future cloth configurations from previous states. For one-step prediction, this can be written as

$$X^{(t+1)} = \Phi(S^{(t)}), \tag{1}$$

where Φ denotes the learned transition function.

More generally, the simulator should be able to predict a full rollout over multiple time steps. For a horizon $k \in \mathbb{N}^*$, we write

$$X^{(t+k)} = \Phi_k(S^{(t)}), \tag{2}$$

where Φ_k denotes the result of applying the learned dynamics for k steps.

Mesh-based learned simulators usually apply the recurrent transition directly in vertex space, for example through graph message passing on the mesh connectivity [5, 6, 8]. In this setting, the learned transition is repeatedly applied to the mesh graph itself. As a result, the recurrent computation depends on the number of vertices and edges in the simulated mesh.

4. Latent UV-Atlas Dynamics

The proposed simulator performs the recurrent dynamics in a latent UV-atlas space rather than directly on mesh vertices. Following the rollout formulation introduced in the previous section, we decompose the learned simulator into three parts:

$$\Phi_k = D \circ P^k \circ E. \quad (3)$$

Here, E is an encoder that maps the input cloth state to a latent atlas state, P is a latent transition model applied recurrently, and D is a decoder that maps the predicted latent state back to mesh vertex positions.

Using latent variables, this can be written as

$$\begin{aligned} Z^{(t)} &= E(S^{(t)}), \\ Z^{(t+1)} &= P(Z^{(t)}, Z^{(t-1)}), \end{aligned}$$

and, after k latent transition steps,

$$X^{(t+k)} = D(Z^{(t+k)}, S^{(0)}).$$

The role of $S^{(0)}$ in the decoder is to provide time-independent information such as the rest configuration and the node types. The key point of this formulation is that the recurrent part of the simulator is applied in latent atlas space rather than directly on the mesh vertices. This follows the same general motivation as recent latent-space cloth simulators, which aim to reduce the dependence of temporal dynamics on the mesh resolution [11]. In our case, the repeated transition P^k is not tied directly to the number of vertices in the output mesh.

4.1. UV-Atlas Representation

The formulation introduced in Eq. (2) defines the cloth state on mesh vertices. Since our goal is to reduce the dependence of the learned dynamics on the mesh resolution, we first map this state to a fixed-resolution UV-atlas domain and apply the recurrent dynamics in this representation.

We assume that the cloth surface is equipped with a UV parametrisation, which provides a two-dimensional material-space domain attached to the cloth surface. From the mesh state, we construct an atlas observation tensor

$$A^{(t)} \in \mathbb{R}^{H \times W \times C_{\text{in}}},$$

where H and W are the atlas resolution and C_{in} is the number of input channels. Each atlas texel stores cloth information obtained through the UV parametrisation. The input channels include current position, velocity, reference position, node type encoded as a one-hot vector, and Green strain components. Vertex quantities are rasterised using barycentric interpolation, while face quantities are assigned using the nearest face, as shown in Fig. 1.

The Green strain components provide a local measure of deformation in material coordinates. This makes them consistent with the UV-atlas representation, which is also defined on the cloth material domain. They also provide the atlas representation with local stretching information that

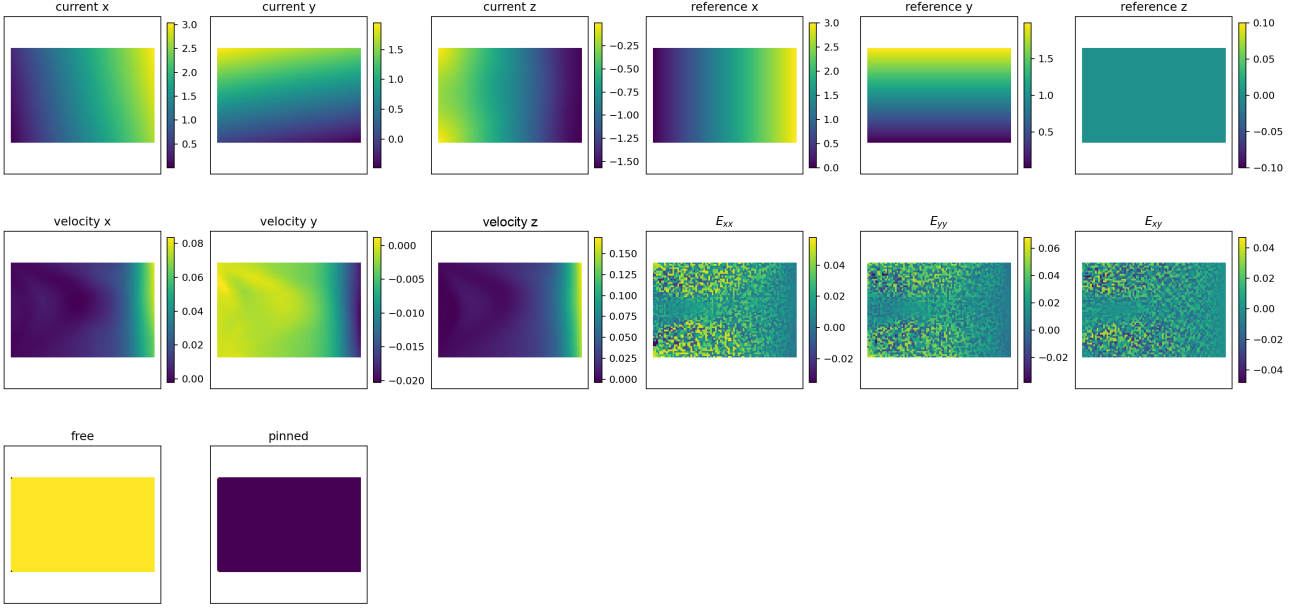


Figure 1: Unnormalised input atlas channels from an ARCSim flag simulation under constant wind and gravity.

would otherwise be explicit in a mesh graph through edge lengths and relative displacements, as in MeshGraphNets-style simulators [5].

The resulting atlas representation changes the role of the mesh in the simulator. The mesh is used to construct the input atlas and to define the output query points, but it is not the domain on which the recurrent dynamics are repeatedly applied. Since a UV atlas may contain seams between charts, standard convolutions do not always reflect the true surface neighbourhood. We therefore use cross-atlas convolutions to preserve local continuity in material space across chart boundaries [13].

This formulation reduces the dependence of the recurrent model on the number of mesh vertices. It does not make the method fully discretisation-independent, since the atlas resolution is fixed and the representation depends on the quality of the UV parametrisation. The atlas resolution must be chosen to balance computational performance and representation capacity. However, if two meshes describe the same cloth surface with compatible UV coordinates, they can be mapped to the same atlas domain and queried from the same latent representation.

4.2. Model Architecture

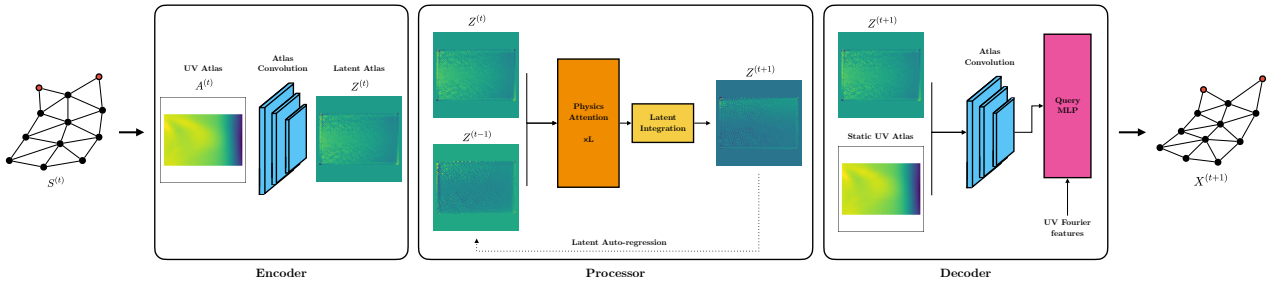


Figure 2: Overview of the proposed UV-atlas latent simulator. The mesh state is first rasterised into a fixed-resolution UV atlas, encoded into a latent atlas representation, evolved in latent space, and decoded back to mesh vertex positions through UV queries.

The proposed simulator follows the factorisation introduced in Eq. (3). The mesh state is first mapped

to a UV-atlas observation tensor, encoded into a latent atlas state, evolved autoregressively in latent space, and finally decoded back to mesh vertex positions. The recurrent part of the model is therefore applied to a fixed-resolution atlas representation rather than directly on the mesh vertices. The overall architecture is shown in Fig. 2.

Encoder. The encoder maps the atlas observation tensor $A^{(t)}$ to a latent atlas state

$$Z^{(t)} = E(A^{(t)}), \quad Z^{(t)} \in \mathbb{R}^{H \times W \times C_z}.$$

The input channels are first normalised, then processed with a cross-atlas convolution followed by multiple residual cross-atlas blocks. This converts the UV-atlas observation into a latent atlas map.

For the next step, we need $Z^{(t-1)}$. When we start from a static position we can use the same vector twice and then keep track of the previous and current ones.

To initialise the second-order latent update, the first rollout step uses $Z^{(t-1)}$ when the cloth starts from rest. During the rollout, both the previous and current latent states are stored and updated autoregressively.

Processor. The dynamics are learned directly in latent atlas space. Given two consecutive latent states $Z^{(t)}$ and $Z^{(t-1)}$, the processor receives the current latent state together with a latent velocity term $Z^{(t)} - Z^{(t-1)}$. This input is passed through convolutional pre-processing composed of cross-atlas convolution blocks, followed by L physics-attention blocks. This processor is inspired by recent Transformer-based learned simulators that combine local mesh or spatial processing with global physics-attention updates [9, 10]. The output is then processed by convolutional post-processing. The processor predicts a latent acceleration $a^{(t)}$, which is used to update the latent atlas with

$$Z^{(t+1)} = Z^{(t)} + \left(Z^{(t)} - Z^{(t-1)} \right) + a^{(t)}. \quad (4)$$

This gives the rollout an inertial structure similar to Verlet integration, while keeping the recurrent state entirely in latent space. During simulation, the processor is applied repeatedly, and decoded positions are not fed back into the model.

Decoder and Query Mechanism. The decoder maps the predicted latent atlas state back to mesh vertex positions. Before decoding, the latent state is concatenated with a static atlas context containing time-independent information such as the reference position and the node type. This combined atlas is processed with a cross-atlas convolution to produce a lower-dimensional feature map used for decoding.

The mesh vertices are then treated as query points in the UV domain. For each vertex i , the decoder samples the atlas feature map at the corresponding UV coordinate u_i using bilinear interpolation. The sampled feature is concatenated with a Fourier encoding of the UV coordinate and passed to an MLP, which predicts the 3D position of the vertex:

$$\hat{x}_i^{(t+1)} = D \left(\text{sample}(Z^{(t+1)}, u_i), \gamma(u_i) \right).$$

This query-based decoder allows the same latent atlas state to be decoded on different mesh samplings, as long as they share a compatible UV parametrisation.

4.3. Training Strategy

During training, rollouts are performed in latent space, while decoded positions are used only to compute losses. They are not fed back into the model.

To improve long-horizon stability, we use a rollout curriculum. Training starts with short rollouts, and the maximum rollout horizon is progressively increased until reaching the maximum allowed by the dataset. This gradually exposes the model to the error accumulation that appears during autoregressive prediction, which is a common difficulty in learned physical simulators [5].

The training objective combines supervised losses with physics-based regularisation. The supervised terms compare both the predicted latent state and the decoded vertex positions to the target trajectory. We also add a latent acceleration loss, which encourages the predicted acceleration $a^{(t)}$ to match the acceleration induced by the target latent trajectory:

$$a_{\text{target}}^{(t)} = E(S^{(t+1)}) - Z^{(t)} - (Z^{(t)} - Z^{(t-1)}).$$

This term is consistent with the second-order latent update introduced in Eq. (4). In practice, gradients are stopped through the target latent state $E(S^{(t+1)})$ when computing this loss.

The physics-based terms are computed on the decoded mesh positions. In the current formulation, we follow the use of elastic and bending energies in physics-based cloth objectives [12].

The loss at each rollout step is written as:

$$\mathcal{L}^{(t)} = \mathcal{L}_{\text{latent}}^{(t)} + \mathcal{L}_{\text{position}}^{(t)} + \mathcal{L}_{\text{physics}}^{(t)},$$

The latent loss combines a latent state loss and a latent acceleration loss:

$$\mathcal{L}_{\text{latent}}^{(t)} = \mathcal{L}_{\text{latent-acc}}^{(t)} + \mathcal{L}_{\text{latent-state}}^{(t)},$$

The physics loss is derived from the optimisation formulation of the equation of motion:

$$\mathcal{L}_{\text{physics}}^{(t)} = \mathcal{L}_{\text{strain}}^{(t)} + \mathcal{L}_{\text{bending}}^{(t)} + \mathcal{L}_{\text{inertia}}^{(t)} + \mathcal{L}_{\text{gravity}}^{(t)}.$$

The final objective is evaluated over the rollout rather than only at the first predicted step. Later steps are weighted slightly more strongly to encourage stability after errors have accumulated:

$$\mathcal{L} = \frac{\sum_{k=1}^n \gamma^k \mathcal{L}^{(t+k)}}{\sum_{k=1}^n \gamma^k}.$$

For memory efficiency, long rollouts are split into a detached prefix and a shorter gradient-tracked tail. The detached prefix allows the model to reach later rollout states without storing all intermediate activations, while the final steps remain differentiable. This strategy was necessary to fit the training process on a single NVIDIA Tesla P100 GPU with 16 GB of memory; 8 CPU cores were used for data-loading workers. With this hardware setup, a full training run took up to 24 hours.

5. Experiments

To evaluate the proposed model, we compare it against state-of-the-art learned simulators including MeshGraphNet (MGN) [5] and MeshGraphNet-Transformer (MGN-T) [10]. The goal of these experiments is to study how the proposed UV-atlas latent formulation behaves under different mesh samplings, and how its sampling robustness compares to mesh-based baselines.

5.1. Implementation Details

Architecture. We use an input atlas of size $96 \times 96 \times 14$. The 14 input channels correspond to the current position (3 channels), rest position (3), velocity (3), Green strain tensor components (3), and node type encoded as a one-hot vector (2). The number of hidden channels is fixed to 64.

An atlas convolution is implemented as a 3×3 convolution following the cross-atlas convolution of [13]. The residual atlas convolution block is defined using the sigmoid linear unit (SiLU) activation function as

$$\text{AtlasConv}(64, 32) \rightarrow \text{SiLU} \rightarrow \text{AtlasConv}(32, 64) \rightarrow \text{skip addition}.$$

The encoder consists of one atlas convolution followed by two residual atlas convolution blocks.

The processor first concatenates the previous latent states with the latent velocity, and passes the resulting tensor through a pre-processing block composed of an atlas convolution followed by a residual atlas convolution. The physics-attention module follows the idea of Transolver [9] and the implementation strategy of MGN-T. We use 64 physical slices and $L = 4$ multi-head attention blocks, with 4 heads each.

The output of the physics-attention module is then passed through a post-processing block similar to the pre-processing one. This block predicts a normalised latent acceleration. The normalisation constants are accumulated during training using an EMA strategy. At inference, the predicted acceleration is denormalised and integrated with a Verlet-style update to obtain the next latent state.

The decoder starts with an atlas convolution applied to the concatenation of the predicted latent map and the static input maps. The resulting feature map is sampled at the UV position of each output vertex using bilinear interpolation. The UV coordinate is also encoded with Fourier features using four frequencies, together with the normalised coordinates, giving 18 features in total. Finally, an MLP with four hidden layers predicts the 3D position from the sampled atlas feature and the Fourier-encoded UV coordinate.

Training. The model is trained with the Adam optimiser using an initial learning rate of 3×10^{-4} , exponentially decayed to 1×10^{-6} over 90,000 steps. We use a rollout curriculum: training starts with single-step prediction and progressively increases the rollout horizon up to 16 steps over the 180,000 training steps.

To improve robustness during autoregressive rollout, we add Gaussian noise to the latent map during training, with standard deviation 1×10^{-2} . For the exponential weighting of the losses along the rollout, we use $\gamma = 1.03$.

5.2. Dataset

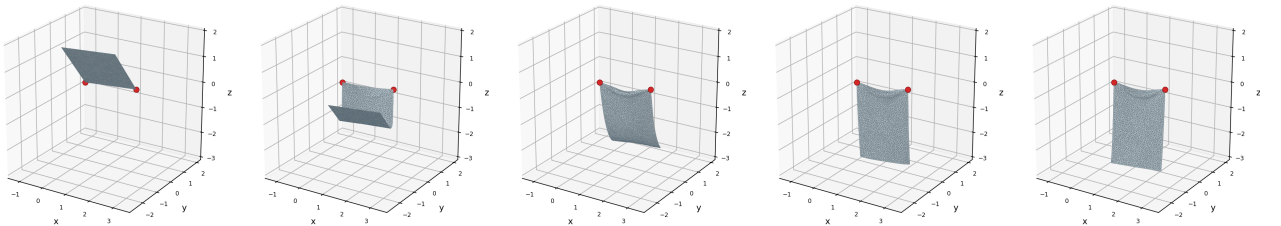


Figure 3: Example simulation from the dataset, sampled every 20 frames.

To train our model and the baselines, we generated a dataset using ARCSim [3, 4]. The training simulations use a fixed mesh sampling generated with Poisson disk sampling followed by Delaunay triangulation. The simulated cloth is a rectangular flag with aspect ratio 3×2 . We use a minimum distance of 0.05 between vertices, resulting in approximately 1.6k vertices per mesh.

For this comparison, we use a simple setting with gravity only and two pinned vertices. The dataset contains 20 simulations of 100 steps each. During training, 10% of the dataset is used for validation. See Fig. 3 for an example of one simulation in the dataset.

To evaluate the behaviour of the models under different mesh samplings, we also generate lower-resolution test meshes using both Poisson and uniform sampling. Table 1 reports detailed information on the test meshes. These meshes follow the same underlying motions as the corresponding high-resolution simulations: the ground-truth trajectories are transferred to the new vertices by barycentric interpolation from the original simulated mesh. This allows us to evaluate the models on different discretisations while keeping the physical motion comparable.

Sampling	Spacing	Vertices	V rel.	Faces	F rel.
Original (Poisson)	0.050	1642	-	3082	-
Poisson	0.075	756	46%	1376	45%
Poisson	0.100	434	26%	766	25%
Poisson	0.200	130	8%	208	8%
Poisson	0.500	27	2%	32	1%
Uniform	0.075	1148	70%	2160	70%
Uniform	0.100	651	40%	1200	39%
Uniform	0.200	176	11%	300	10%
Uniform	0.500	35	2%	48	2%

Table 1: Mesh statistics for the original training sampling and the lower-resolution test samplings. The relative size column reports vertex and face percentages with respect to the original Poisson disk mesh.

5.3. Results and Discussion

Note on MGN-T. The MGN-T architecture [10] was originally designed for solid mechanics. However, since MeshGraphNets [5] have been shown to work on both cloth simulation and solid mechanics, we adapted MGN-T to the cloth setting in the same spirit. In practice, this baseline performed better than standard MGN when trained under the same conditions. The main modifications are the input and output representations: instead of using the solid-mechanics state variables from the original paper, we provide cloth-specific per-vertex features similar to those used by MGN and our model, and predict 3D vertex accelerations.

We evaluate the models using qualitative rollouts and mean vertex error (MVE) over time. The objective is to measure how the error changes when the same motion is evaluated on different mesh samplings. This is the main point of the experiment, since the proposed method is designed to reduce the dependence of the recurrent dynamics on the output mesh discretisation.

Fig. 4 shows the evolution of the MVE during rollout for the different sampling conditions. The graph-based baselines have the lowest error on the original validation mesh, but their error increases substantially when the mesh is undersampled. This is especially visible for MGN, whose curves grow rapidly as the vertex count decreases. MGN-T is more stable than MGN, which is consistent with its use of physics-attention to model nonlocal interactions, but it still remains sensitive to the change in mesh sampling. In contrast, the curves for our model are much closer to each other across the different samplings.

This trend is confirmed by the robustness summary in Table 2. Our method obtains the lowest mean MVE across all evaluated samplings, with a mean MVE of 0.164, while both baselines have a higher mean error. This value is computed across all samplings and all validation simulations, corresponding to 18 rollouts of 25 steps each. We also observe that MGN-T performs better than MGN on all reported metrics, as already visible in Fig. 4. Our model also has the lowest coefficient of variation, with a value of 0.120, which is 3.7 times lower than the next lowest value, obtained by MGN-T. This indicates that the proposed UV-atlas latent formulation is less sensitive to changes in mesh sampling.

Table 3 gives a more detailed comparison on the Poisson disk samplings. On the original validation

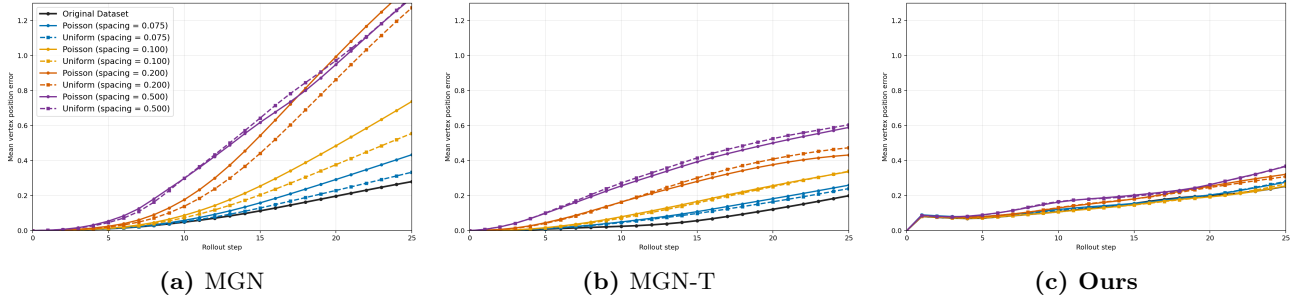


Figure 4: Mean vertex position error evolution over rollout time for the different mesh samplings.

Metric	MGN	MGN-T	Ours
Mean MVE	0.345	0.199	0.164
Std.	0.169	0.089	0.020
CV	0.491	0.446	0.120

Table 2: Sampling robustness summary across all evaluated mesh samplings. Lower standard deviation and coefficient of variation (CV) indicate lower sensitivity to changes in mesh sampling.

mesh, MGN-T obtains the lowest error, while our model is less accurate, with an average MVE of 0.143 and a final-step MVE of 0.254. This suggests that the graph-based baseline fits the training discretisation more accurately.

The trend changes when the mesh is undersampled. As the Poisson disk spacing increases, the error of the graph-based models grows more strongly, while our method becomes the best-performing model on all undersampled Poisson meshes. For example, from the original mesh to Poisson spacing 0.20, the average MVE of our model increases only from 0.143 to 0.174. This supports the intended behaviour of the UV-atlas representation: the recurrent dynamics are computed in a fixed-resolution latent atlas, so changing the output mesh mainly changes the query points used by the decoder.

Sampling	MGN		MGN-T		Ours	
	Avg.	Final	Avg.	Final	Avg.	Final
Original	0.106	0.280	0.065	0.199	0.143	0.254
Poisson 0.10	0.253	0.737	0.143	0.337	0.141	0.256
Poisson 0.20	0.510	1.404	0.225	0.432	0.174	0.322
Poisson 0.50	0.537	1.328	0.319	0.589	0.192	0.367

Table 3: Mean vertex error on Poisson disk samplings. For each sampling, bold values indicate the best model.

The qualitative rollouts in Fig. 5 and Fig. 6 are consistent with the quantitative results. On the original validation mesh, all models reproduce the global falling motion of the flag, with MGN-T producing the most accurate trajectory according to the MVE. On the undersampled Poisson mesh, the graph-based baselines remain plausible at the beginning of the rollout, but their predictions become more affected by the change in discretisation. Our method preserves a more similar motion across samplings, which is consistent with the lower variation reported in Table 2.

Overall, the results do not show that the proposed method is always the most accurate. Instead, they show that it is more robust to mesh resampling. This is an important distinction. MGN and MGN-T can exploit the exact graph structure seen during training and therefore achieve lower error on the original mesh. Our model compromises on mesh-specific accuracy, but gains stability when the output mesh changes. This supports the main motivation of the method: separating the recurrent dynamics from the number of vertices in the output mesh is a promising direction for sampling-agnostic cloth

simulation.

6. Conclusion

We presented a UV-atlas latent simulator for cloth dynamics. Instead of applying the recurrent transition directly to mesh vertices, the cloth state is mapped to a fixed-resolution UV atlas, evolved in latent space, and decoded back to mesh positions through UV queries. This separates the recurrent dynamics from the number of vertices in the output mesh while keeping the representation aligned with the cloth material domain.

Experiments on different mesh samplings show that the proposed method is more robust to resampling than mesh-based baselines. MGN and MGN-T achieve lower error on the original validation mesh, but their performance degrades more strongly when the mesh resolution changes. Our model is less accurate on the training discretisation, but maintains a more stable error across Poisson and uniform lower-resolution samplings. This supports UV-atlas latent dynamics as a promising direction for sampling-agnostic cloth simulation.

Limitations and Future Work. The current evaluation is limited to a simple gravity-driven flag setup with two pinned vertices. This setting is useful for isolating the effect of mesh resampling, but it does not include contacts, self-collisions, body interactions, or garment-like motion. A major direction for future work is therefore to extend the model with explicit collision handling, either through additional collision-aware inputs, physics-based collision losses, or post-processing constraints.

Another important extension would be to condition the latent dynamics on external constraints or controls. In the current setup, the pinned vertices are fixed, so the processor does not need to react to moving boundary conditions. For more general cloth scenarios, the model should be able to handle animated pinned vertices, moving anchors, or interactive objects. This could be done by providing time-dependent constraint maps in the UV atlas, adding object or body features to the processor, or conditioning the latent transition on external motion signals.

The method is also not fully discretisation-independent, since it still depends on the atlas resolution and the quality of the UV parametrisation. Seams, chart distortion, and limited texel resolution can affect the latent representation. Improving the decoder, increasing the latent atlas resolution, and training on more diverse cloth motions and materials are important directions for improving both accuracy and generalisation.

References

- [1] David Baraff and Andrew P. Witkin. “Large Steps in Cloth Simulation”. In: *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, 1998, pp. 43–54. DOI: [10.1145/280814.280821](https://doi.org/10.1145/280814.280821).
- [2] Robert Bridson, Ronald Fedkiw and John Anderson. “Robust Treatment of Collisions, Contact and Friction for Cloth Animation”. In: *ACM Transactions on Graphics* 21.3 (2002), pp. 594–603. DOI: [10.1145/566654.566623](https://doi.org/10.1145/566654.566623).
- [3] Rahul Narain, Armin Samii and James F. O’Brien. “Adaptive Anisotropic Remeshing for Cloth Simulation”. In: *ACM Transactions on Graphics* 31.6 (2012), 147:1–147:10. DOI: [10.1145/2366145.2366171](https://doi.org/10.1145/2366145.2366171).
- [4] Rahul Narain, Tobias Pfaff and James F. O’Brien. “Folding and Crumpling Adaptive Sheets”. In: *ACM Transactions on Graphics* 32.4 (2013), 51:1–51:8. DOI: [10.1145/2461912.2462010](https://doi.org/10.1145/2461912.2462010).

- [5] Tobias Pfaff et al. “Learning Mesh-Based Simulation with Graph Networks”. In: *International Conference on Learning Representations*. 2021. URL: https://openreview.net/forum?id=roNqYLO_XP.
- [6] Artur Grigorev, Michael J. Black and Otmar Hilliges. “HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 16965–16974. DOI: [10.1109/CVPR52729.2023.01627](https://doi.org/10.1109/CVPR52729.2023.01627).
- [7] Zhouyingcheng Liao, Sinan Wang and Taku Komura. “SENC: Handling Self-Collision in Neural Cloth Simulation”. In: *European Conference on Computer Vision*. Springer, 2024, pp. 385–402. DOI: [10.1007/978-3-031-72673-6_21](https://doi.org/10.1007/978-3-031-72673-6_21).
- [8] Emmanuel Ian Libao et al. “MeshGraphNetRP: Improving Generalization of GNN-Based Cloth Simulation”. In: *ACM SIGGRAPH Conference on Motion, Interaction and Games*. 2023, 5:1–5:7. DOI: [10.1145/3623264.3624441](https://doi.org/10.1145/3623264.3624441).
- [9] Haixu Wu et al. “Transolver: A Fast Transformer Solver for PDEs on General Geometries”. In: *International Conference on Machine Learning*. 2024. arXiv: [2402.02366](https://arxiv.org/abs/2402.02366) [cs.LG]. URL: <https://arxiv.org/abs/2402.02366>.
- [10] M. M. Iparraguirre et al. “MeshGraphNet-Transformer: Scalable Mesh-Based Learned Simulation for Solid Mechanics”. In: *arXiv preprint arXiv:2601.23177* (2026). arXiv: [2601.23177](https://arxiv.org/abs/2601.23177) [cs.LG]. URL: <https://arxiv.org/abs/2601.23177>.
- [11] Yu Zhang et al. “ClothTransformer: Unified Latent-Space Transformers for Scalable Cloth Simulation”. In: *arXiv preprint arXiv:2605.27852* (2026). arXiv: [2605.27852](https://arxiv.org/abs/2605.27852) [cs.GR]. URL: <https://arxiv.org/abs/2605.27852>.
- [12] Igor Santesteban, Miguel A. Otaduy and Dan Casas. “SNUG: Self-Supervised Neural Dynamic Garments”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 8130–8140. DOI: [10.1109/CVPR52688.2022.00797](https://doi.org/10.1109/CVPR52688.2022.00797).
- [13] Shiwei Li et al. “Cross-Atlas Convolution for Parameterization Invariant Learning on Textured Mesh Surface”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 6136–6145. DOI: [10.1109/CVPR.2019.00630](https://doi.org/10.1109/CVPR.2019.00630).
- [14] Igor Santesteban, Miguel A. Otaduy and Dan Casas. “Learning-Based Animation of Clothing for Virtual Try-On”. In: *Computer Graphics Forum*. Vol. 38. 2. 2019, pp. 355–366. DOI: [10.1111/cgf.13643](https://doi.org/10.1111/cgf.13643).
- [15] Igor Santesteban et al. “Self-Supervised Collision Handling via Generative 3D Garment Models for Virtual Try-On”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 11763–11773. DOI: [10.1109/CVPR46437.2021.01159](https://doi.org/10.1109/CVPR46437.2021.01159).
- [16] Hugo Bertiche, Meysam Madadi and Sergio Escalera. “Neural Cloth Simulation”. In: *ACM Transactions on Graphics* 41.6 (2022), pp. 1–14. DOI: [10.1145/3550454.3555491](https://doi.org/10.1145/3550454.3555491).
- [17] Yudi Li et al. “N-Cloth: Predicting 3D Cloth Deformation with Mesh-Based Networks”. In: *Computer Graphics Forum* 41.2 (2022), pp. 547–558. DOI: [10.1111/cgf.14493](https://doi.org/10.1111/cgf.14493). arXiv: [2112.06397](https://arxiv.org/abs/2112.06397) [cs.GR].
- [18] Yang Zhang, Kailuo Yu and Xinyu Zhang. “FastClothGNN: Optimizing Message Passing in Graph Neural Networks for Accelerating Real-Time Cloth Simulation”. In: *Graphical Models* 139 (2025), p. 101273. DOI: [10.1016/j.gmod.2025.101273](https://doi.org/10.1016/j.gmod.2025.101273).
- [19] Alvaro Sanchez-Gonzalez et al. “Learning to Simulate Complex Physics with Graph Networks”. In: *International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 8459–8468. URL: <http://proceedings.mlr.press/v119/sanchez-gonzalez20a.html>.
- [20] Meire Fortunato et al. “Multiscale MeshGraphNets”. In: *arXiv preprint arXiv:2210.00612* (2022). arXiv: [2210.00612](https://arxiv.org/abs/2210.00612).

- [21] Yadi Cao et al. “Efficient Learning of Mesh-Based Physical Simulation with Bi-Stride Multi-Scale Graph Neural Network”. In: *International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 3541–3558. URL: <https://proceedings.mlr.press/v202/cao23a.html>.
- [22] Huakun Luo et al. “Transolver++: An Accurate Neural Solver for PDEs on Million-Scale Geometries”. In: (2025). arXiv: [2502.02414](https://arxiv.org/abs/2502.02414) [cs.LG]. URL: <https://arxiv.org/abs/2502.02414>.

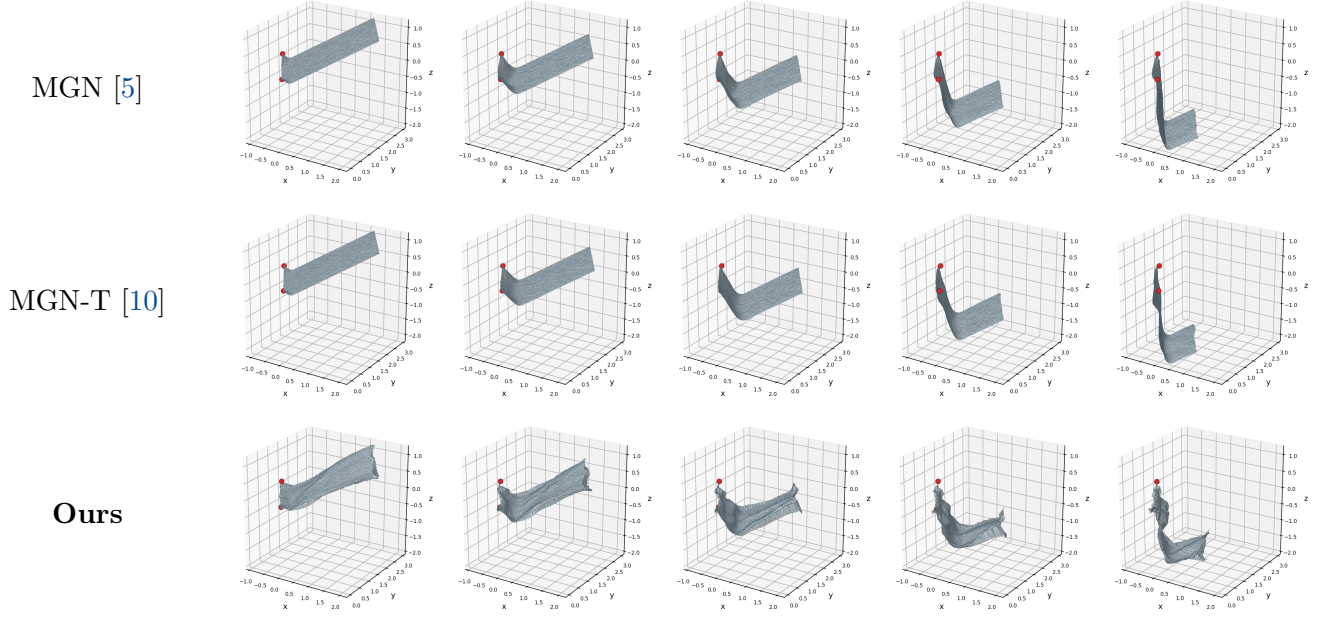


Figure 5: Qualitative rollout comparison on the validation set of the original dataset. The mesh uses the same Poisson disk sampling as the training simulations, with spacing 0.05.

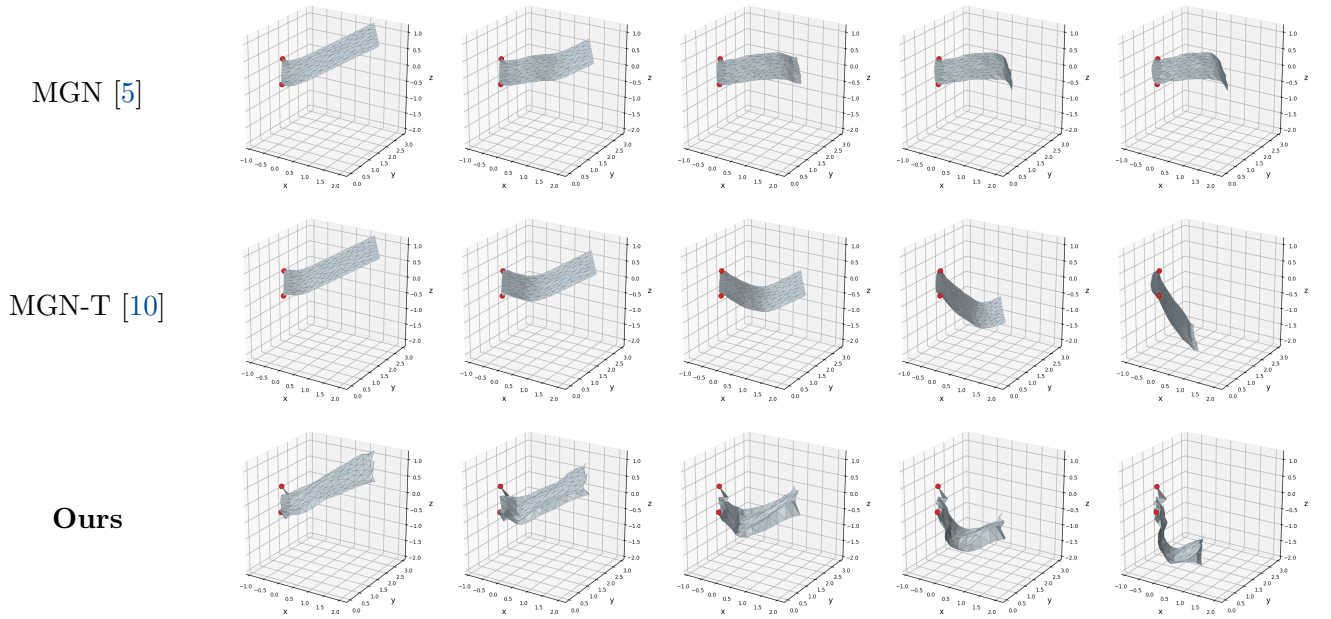


Figure 6: Qualitative rollout comparison on an undersampled Poisson disk mesh. The mesh uses spacing 0.20.